

Monitoring Analytics

Plataforma Intellix — auditoría de todo lo que mide, loguea, traza y alerta en **producción** (VPS 200.58.109.110, /opt/mutualyf, rama main @ d324606).

Relevado el 2026-08-30 contra el código real y el VPS en vivo (docker stats, Prometheus API, Loki API, Jaeger API, Alertmanager, PostgreSQL, Redis, crons). Ningún número de este documento es estimado: cada uno sale de un comando que se indica en el anexo.

1. Resumen ejecutivo

14

contenedores de observabilidad/admin corriendo

2,7 GB

RAM que consumen (2,0 GB es Jaeger solo)

~9 %

de UN core (promedio 7 días) — irrelevante en 16 vCPU

1,5 GB

disco (Prometheus 1,4 GB · Loki 1,2 MB · logs Docker 95 MB)

El sistema tiene **cuatro capas de medición** que se construyeron en momentos distintos y hoy se solapan: (1) métricas Prometheus + 22 alertas por email, (2) logs centralizados en Loki + buffer de errores en Redis, (3) trazas OpenTelemetry a Jaeger, y (4) analítica de negocio en PostgreSQL (usage_events, consultas_log, audit_log) que alimenta las pantallas del panel. La capa 1 y la capa 4 **funcionan y se usan**. Las capas 2 y 3 **están encendidas pero no sirven** en su estado actual: Loki recibe solo los logs JSON del backend (1.461 líneas en 7 días) y Jaeger guarda en RAM 2 GB de trazas de /health y /metrics.

Hallazgo de seguridad (arreglar primero): <https://intellix.com.ar/metrics> responde **HTTP 200 sin autenticación**. Expone ids de tenants, rutas de la API, contadores de uso y el modelo LLM. Es un location /metrics en nginx/nginx.prod.conf (línea 152 en el bloque HTTPS, 274 en HTTP) que hace proxy al backend. Prometheus lo scrapea por la red interna de Docker: **no necesita el location público**. En staging (dev.intellix.com.ar) devuelve 307, o sea que allí sí está tapado.

Los 10 hallazgos, ordenados por impacto

#	Hallazgo	Evidencia real	Estado
1	/metrics público sin auth	curl https://intellix.com.ar/metrics → 200 con ia_queries_total{tenant_id="mutualyf"}	Bug seguridad
2	Contadores ia_* inconsistentes: 4 workers uvicorn, cada uno con su registro; cada scrape cae en un worker distinto	3 scrapes seguidos: ia_queries_total cached = 9 → 10 → 9, ia_groq_requests_total = 588 → 587 → 588	Bug
3	Loki casi vacío: drop_malformed: true descarta toda línea no-JSON (nginx, postgres, celery, satélites, uvicorn access)	Contenedores presentes en Loki (7 días): solo ia_backend (1.461 líneas) y ia_jaeger (39). Volumen: 1,2 MB. Labels level/tenant_id: vacíos	Bug
4	Jaeger: 2,0 GB RAM y creciendo (1,79 GB hace 8 días), almacenamiento en memoria sin tope, y las trazas son basura	200 trazas/24 h: GET /health (147), GET /metrics (41), POST (12) sin ruta ni atributos	Bug + desperdicio
5	cAdvisor sigue corriendo y scrapeado aunque se reemplazó por Telegraf el 20/08	157 series en Prometheus, 0 con name de contenedor (inútiles). Versión v0.52.1 levantada hace 9 días	Duplicado
6	98,6 % de los logs del backend son ruido del scrape	24 h: 5.846 líneas, 5.765 son "GET /metrics" 200 (una cada 15 s). /health sí se filtra, /metrics no	Ruido
7	Gauge ia_active_tenants nunca se setea; se muestra igual en el panel superadmin ("Organizaciones activas")	Vale 0.0 siempre; no hay ningún .set() en el código	Métrica muerta
8	Celery worker no expone métricas: ingestas y quality gate (ia_ingest_total, ia_pipeline_duration_ms, ia_quality_gate_total) se incrementan en un proceso que Prometheus no scrapea	Ningún target celery en prometheus.yml; el dashboard "Documentos ingestados por día" siempre da 0	Métricas ciegas
9	El repo en el VPS tiene 6 archivos modificados y 5 sin trackear que main no tiene (telegraf, dashboards vps-*, reglas contenedores, script de backup)	git status en /opt/mutualyf; git cat-file main:observability/telegraf/telegraf.conf → no existe	Drift
10	pgAdmin y Portainer llevan 3 meses arriba sin uso (ambos solo loopback)	150 MB RAM, 0 % CPU; Portainer con wizard vencido (ver comentario en compose)	Sin uso

Lo que está bien y hay que conservar: las 22 reglas de alerta cargan y evalúan (12 targets up, 0 alertas activas hoy); Alertmanager envió **10 emails** vía Resend con **0 fallos**; los backups publican métricas al textfile_collector y tienen dead-man switch; el error buffer en Redis le da al superadmin los errores sin abrir Grafana; usage_events y consultas_log (con la señal del trust gate desde la migración 055) son la fuente confiable de analítica de negocio; la retención está acotada (Prometheus 90 d, Loki 30 d, consultas_log 180 d, tool_call_audit 90 d, logs Docker 10 MB × 3).

2. Inventario: qué corre hoy en producción

Consumo medido con `docker stats` (instantáneo) y promedio de 7 días desde Telegraf (`docker_container_cpu_usage_percent`, `docker_container_mem_usage`). CPU % es sobre **un** core (16 disponibles).

Contenedor	Imagen	Para qué sirve	Cómo se ve	CPU 7d	RAM 7d	Up	Veredicto
ia_prometheus	prom/prometheus v2.52.0	Guarda métricas (15 s), evalúa 22 alertas, retención 90 d	Grafana; panel superadmin (<code>core/prometheus.py</code>); túnel :9090	0,61 %	143 MB	9 d	Núcleo
ia_alertmanager	prom/alertmanager v0.27.0	Agrupar alertas y manda email (Resend) a 2 destinatarios	Email; tab "Alertas activas" del superadmin	0,13 %	17 MB	2 m	Núcleo
ia_grafana	grafana 10.4.3	4 dashboards provisionados, datasources Prometheus + Loki	<code>http://200.58.109.110/grafana/</code> (nginx) o túnel :3001; usuarios admin y equipo (nunca entró)	0,13 %	55 MB	9 d	Núcleo
ia_node_exporter	node-exporter v1.8.0	CPU/RAM/disco del host + métricas de backup (textfile)	Dashboards vps-*, alertas host y backups	0,0 %	14 MB	2 sem	Núcleo
ia_docker_exporter	telegraf 1.31-alpine	CPU/RAM/red por contenedor (reemplazo de cAdvisor)	Dashboards vps-estado / vps-historico; alertas de saturación	3,03 %	76 MB	9 d	Núcleo
ia_blackbox_exporter	blackbox v0.25.0	Sondea desde afuera /health de prod, staging y 3 satélites; vencimiento SSL	Alertas PlatformDown / SatelliteDown / SslCert	1,69 %	23 MB	3 m	Núcleo
ia_postgres_exporter	postgres-exporter v0.15.0	Estado de PG (conexiones, tamaño, cache hit, deadlocks)	Panel superadmin > Métricas; alerta PostgresDown	0,0 %	18 MB	2 m	Núcleo
ia_redis_exporter	redis_exporter v1.62.0	Estado de Redis (memoria, hit rate, evictions, keys por DB)	Panel superadmin > Métricas; alerta RedisDown	0,0 %	14 MB	3 m	Núcleo
ia_loki	loki 3.0.0	Almacén de logs, retención 30 d con compactor	Grafana Explore; dashboards "Logs por contenedor" y "Overview"	1,17 %	72 MB	3 m	Recibe casi nada
ia_promtail	promtail 3.0.0	Lee logs de todos los contenedores vía socket Docker y los manda a Loki	—	1,36 %	147 MB	9 d	Descarta lo no-JSON
ia_jaeger	jaeger all-in-one 1.57	Recibe trazas OTLP del backend (10 % sampling), storage en memoria	Túnel :16686	0,89 %	2.008 MB	3 m	Apagar

ia_cadvisor	cadvisor v0.52.1	Métricas por contenedor — no funciona con el containerd-snapshotter de este host	Nada (0 series con nombre)	0,20 %	26 MB	9 d	Borrar
ia_pgadmin	pgadmin4 8.12	UI de PostgreSQL (loopback :5050)	Túnel SSH	0,04 %	123 MB	3 m	Parar
ia_portainer	portainer-ce 2.21.4	UI de Docker (loopback :9000); wizard de admin vencido	Túnel SSH	0,0 %	27 MB	3 m	Parar

No listado porque no es observabilidad pero sí mide: **ia_backend** (expone /metrics y emite spans/logs), **ia_celery_worker** (incrementa contadores que nadie lee), y el **cron del host** (03:30 mutualyf-pg-backup.sh, 03:45 satelites-backup.sh) que escribe .prom en /var/lib/node_exporter/textfile_collector/. Staging (ia_backend_staging, OTEL_ENABLED=true) NO es scrapeado por Prometheus: solo su /health se sondea con blackbox.

3. Capa 1 — Métricas (Prometheus)

3.1 Fuentes (12 targets, todos up al momento del relevamiento)

Job	Target	Qué aporta	Series
ia_backend	backend:8000/metrics	http_requests_total, http_request_duration_seconds (+ variante _highr) del instrumentator; 8 métricas ia_* propias	Sujeto al bug multiproceso
qdrant	qdrant:6333/metrics	Métricas nativas de Qdrant — ningún dashboard ni alerta las usa	—
redis / postgres	exporters	Estado de las bases; redis_commands_latencies_usec_bucket es la métrica con más series (399) y no se usa	~1.000
docker	docker-exporter:9273 (Telegraf)	CPU/RAM/red por contenedor con container_name y com_docker_compose_project	Usadas en vps-*
cadvisor	cadvisor:8080	157 series de proceso propio, 0 de contenedores	Inútil
node	node-exporter:9100	Host + textfile de backups (mutualyf_backup_*, satelite_backup_*)	128 solo node_cpu_seconds_total
blackbox_health / blackbox_satellites	5 URLs	probe_success, probe_ssl_earliest_cert_expiry	—

TSDB: 6.887 series activas, 41.294 chunks, 1,4 GB en disco. Prometheus recibió 21 GB por red desde que arrancó (scrapes cada 15 s); el postgres-exporter recibió 27 GB de PG. Nada de esto es un problema de capacidad.

3.2 Métricas propias del backend (backend/core/metrics.py)

Métrica	Labels	Dónde se incrementa	¿Llega a Prometheus?	Quién la lee
ia_queries_total	tenant_id, complexity, from_cache	orchestrator.py :134/:188 (cache hit) y :820 (consulta real)	Sí, pero por-worker	Dashboard Overview (QPM, cache hit), panel superadmin ("Consultas totales")
ia_cache_hits_total	tenant_id	orchestrator.py :133/:187	Por-worker	Overview (gauge cache hit rate), superadmin
ia_query_duration_ms	tenant_id, complexity	orchestrator.py :135/:189/:821	Por-worker	Overview (p50/p95/p99)
ia_groq_requests_total	model, status	groq_client.py :194/:220/:428 — el "Groq" es OpenAI gpt-4o-mini	Por-worker	Alertas GroqDownOrNoCredit / GroqDegraded; Overview; superadmin "Servicio de IA"
ia_ingest_total	tenant_id, status	workers/ingest_tasks.py :453 — en Celery	No: Celery no expone /metrics	Overview "Documentos ingestados por día" (siempre 0), superadmin "Ingestas totales"
ia_pipeline_duration_ms	tenant_id	ingest_tasks.py :454 — en Celery	No	Overview "Pipeline de ingesta"
ia_quality_gate_total	status	ingest_tasks.py :456 — en Celery	No	Overview piechart, superadmin
ia_active_tenants	—	Nunca (no hay .set())	Siempre 0	Superadmin "Organizaciones activas"

Por qué "por-worker" importa. uvicorn --workers 4 crea 4 procesos; prometheus_client guarda los contadores en memoria de cada proceso y /metrics devuelve los del proceso que atendió el scrape. Prometheus ve una serie que sube y baja (9 → 10 → 9), rate() la interpreta como reinicios del contador y las alertas de Groq pueden evaluar con un cuarto del tráfico. La solución canónica es el modo multiproceso (PROMETHEUS_MULTIPROC_DIR + MultiProcessCollector) o, más simple para este volumen, correr el instrumentator sobre un solo proceso con contadores compartidos en Redis. Hoy, con ~10 consultas/día, el error es tolerable; a más tráfico no.

3.3 Alertas (22 reglas, 7 grupos) y qué pasó en los últimos 30 días

Grupo	Reglas	Severidad	Disparos 30 d
platform_critical	PlatformDown, BackendDown, HighRagLatencyP95, HighErrorRate5xx, PostgresDown, RedisDown	critical / warning	PlatformDown (prod y staging, ~1,5 min c/u), BackendDown (~1,5 min), HighErrorRate5xx (~2 min) — coinciden con deploys/restarts
groq_llm	GroqDownOrNoCredit, GroqDegraded	critical / warning	0

host_resources	DiskAlmostFull, LowMemory, HighCpuLoad	warning	0
ssl	SslCertExpiringSoon	warning	0
backups	BackupDesactualizado, BackupMetricaAusente, BackupSemanalDesactualizado, BackupDemasiadoChico	critical / warning	0 — último backup OK 30/08 03:30, 1,58 MB, 8 diarios + 6 semanales
satelites	SateliteDown, SateliteBackupDesactualizado, SateliteBackupFallido, SateliteBackupMetricaAusente	warning	SateliteDown (30 s, target viejo /)
contenedores	ContenedorEnTechoDeRAM, ContenedorCPUEstrangulado	warning	0

Alertmanager: **12 alertas firing recibidas, 38 resolved, 10 emails enviados, 0 fallos** (métricas internas alertmanager_notifications_total{integration="email"}). Destinatarios: dos cuentas de Gmail. Inhibición: si PlatformDown está activa, se callan los warnings. Repite cada 4 h mientras siga activa.

Observaciones: (a) HighRagLatencyP95 usa http_request_duration_seconds_bucket{handler=~".*query.*|.*conversation.*"} — con should_group_status_codes y por-worker el p95 es aproximado; (b) HighErrorRate5xx con for: 5m se disparó en restarts porque con ~5 requests/min un solo 5xx ya es >1 %; conviene un mínimo de tráfico (and sum(rate(http_requests_total[5m])) > 0.1); (c) el runbook y las alertas siguen diciendo "Groq" y "console.groq.com" cuando el proveedor es OpenAI.

4. Capa 2 — Logs

4.1 Cómo se generan

- **Backend:** structlog → JSON por línea a stdout en producción (core/logging_config.py). Cada línea lleva request_id (middleware) y, cuando el evento lo incluye, tenant_id. Se silencian sqlalchemy/httpx/torch. Se filtra GET /health del access log de uvicorn, **pero no** GET /metrics: 5.765 líneas/día de ruido (98,6 % del log del backend), en formato texto plano de uvicorn, no JSON.
- **Celery worker/beat:** formato texto de Celery ([2026-08-30 12:56:56,481: INFO/ForkPoolWorker-4] inactivity_check_done ...), **no** JSON.
- **Nginx:** access.log y error.log son symlinks a stdout/stderr, formato main de nginx, **no** JSON. 17 MB (el log más grande del host).
- **Docker:** driver json-file, rotación 10 MB × 3 en todos los servicios ia_*. Total en disco: 95 MB. Bien acotado.

4.2 Dónde terminan (y dónde no)

Destino	Qué contiene realmente	Retención	Cómo se ve
Loki (vía Promtail, docker_sd sin filtro de proyecto)	Solo líneas JSON de structlog de ia_backend (1.461 en 7 d) y 39 de ia_jaeger. Todo lo demás (nginx, postgres, celery, staging, 4 apps satélite) es descartado por drop_malformed: true en el stage json. Contadores de Promtail desde su restart: 2.291 enviadas, 207 descartadas explícitamente (el resto ni cuenta). Los labels level y tenant_id que promete el pipeline están vacíos en Loki.	30 d (compactador activo)	Grafana Explore; dashboard "Logs por contenedor" (muestra 2 contenedores de 41); panel "Logs del backend" del Overview
Error buffer en Redis (platform:recent_errors, DB 1)	Últimos 300 WARNING/ERROR del backend (todos los workers), TTL 7 d. Hoy: 154 ERROR + 146 WARNING. Composición: 150× db_error (columna delivery_status inexistente, del 10/08 — ya corregido pero sigue en el buffer), 76× jwt_decode_failed (tokens vencidos: ruido), 44× query_rewrite_timeout/fallback, 12× model_warmup_failed (intenta cargar el modelo local con EMBEDDING_PROVIDER=openai), 8× quality gate.	300 entradas / 7 d	Superadmin > Monitoreo > Errores (filtro por nivel, agrupado por mensaje); ficha de cada tenant > Salud
docker logs	Todo, crudo, últimos 30 MB por contenedor	Rotación	SSH: docker logs ia_backend

Consecuencia práctica: la instrucción del runbook "Loki → {container="ia_backend"} |= "ERROR"" funciona, pero "ver por qué nginx dio 502" o "qué hizo el worker de ingesta" en Loki **no**: esos logs nunca entraron. Y el buffer de Redis, que sí se mira, está dominado por 150 copias de un error de agosto y por tokens vencidos que no son errores.

5. Capa 3 — Trazas (OpenTelemetry → Jaeger)

OTEL_ENABLED=true y OTEL_SAMPLE_RATIO=0.1 en prod y staging (ambos backends exportan al mismo Jaeger). core/tracing.py instrumenta FastAPI, SQLAlchemy y httpx, y el código agrega spans manuales: query.handle (orchestrator), retrieval.embed, retrieval.qdrant_search, retrieval.parent_expand, retrieval.bm25_rrf (retrieval.py). Eso es lo que *debería* permitir ver "qué paso hace lenta una consulta".

Medición en vivo	Valor
Trazas del servicio en 24 h (tope 200)	200 — spans: GET /health 147, GET /metrics 41, POST 12
Spans raíz de las consultas reales	Se llaman POST a secas, sin http.route/http.target, 765 ms y 2.075 ms — no se sabe qué endpoint son sin abrirlos
Storage	SPAN_STORAGE_TYPE=memory, sin --memory.max-traces → crece hasta que Jaeger reinicie
RAM	2.008 MB hoy, 1.788 MB hace 8 días (+27 MB/día). Es el 5.º contenedor que más RAM usa del host, por encima de Qdrant y Postgres
Uso humano	Solo por túnel SSH :16686. El panel superadmin no lo consulta. El RUNBOOK lo referencia en la sección 5 (latencia)

Veredicto: el 10 % de sampling se aplica a *todas* las requests, y como el 96 % de las requests son health/metrics, las trazas útiles (consultas RAG, ~10/día) son una fracción mínima. Apagarlo (OTEL_ENABLED=false + docker compose stop jaeger) libera 2 GB sin perder nada que se esté usando. Si en el futuro se quiere diagnóstico de latencia, la ruta correcta es excluir /health y /metrics del instrumentador (excluded_urls), samplear al 100 % solo /api/v1/widget/... y poner tope de memoria.

6. Capa 4 — Analítica de negocio en PostgreSQL

Es la capa que **realmente alimenta las pantallas** del producto y la única que sobrevive a reinicios (Prometheus y Redis se resetean; PG no).

Tabla	Quién escribe	Qué guarda	Datos reales (prod)	Retención
public.usage_events	orchestrator._log_usage_event_app (query), groq_client (llm_tokens, fire-and-forget), ingest_tasks (ingest)	tenant_id, event_type, value, created_at	mutualyf: 4.380 consultas, 18,2 M tokens, 68 ingestas (30/05 → 26/08); intellix: 377 consultas, 1,5 M tokens; galo: 12 consultas; staging huérfano (53 filas de mayo). 2,3 MB	Ninguna (crece para siempre; es chica)
tenant_*.consultas_log	orchestrator._log_query	pregunta, hash, latencia, from_cache, y desde la mig. 055: trust_action, trust_lex, trust_best_cos, trust_best_rrf, trust_judge, trust_reason	mutualyf: 2.838 filas, 1,6 MB; últimos 30 d: 350 consultas, p50 3.041 ms, p95 6.370 ms , 52 desde cache (15 %); trust_action: 84 answer, 266 NULL (previas al 23/08, cache, small talk), 0 refuse . Pico de 100 consultas el 25/08 (corrida de suite)	180 d (purga diaria 02:15 en lotes de 500)
tenant_*.audit_log	core/audit.py	actor, acción, recurso, IP	mutualyf 690, intellix 257	Ninguna
tenant_*.tool_call_audit	services/connector_audit.py	Llamadas a conectores	mutualyf 0, intellix 16 (última 21/07) — conectores apagados en prod	90 d
tenant_*.conversaciones	widget / whatsapp / handoff	canal, handoff_requested_at, closed_at, feedback_rating, is_test	mutualyf: 1.121 conversaciones, 118 en 30 d, 34 derivaciones históricas, 0 con feedback (feature oculta por flag)	Ninguna

6.1 Endpoints que agregan estos datos y pantallas que los muestran

Endpoint	Rol	Fuente	Pantalla	Qué se ve
GET /api/v1/metrics?days=7 30 90	admin del tenant	usage_events + consultas_log + documentos + conversaciones + plan	Admin › Métricas › Asistente / Atención (metrics-dashboard.tsx)	Consultas por día, tiempo de respuesta p50/p95, resuelto por caché, cuota del plan, tokens; conversaciones por canal, resueltas por bot vs derivadas, espera hasta operador, resolución promedio, satisfacción (vacía), consultas recientes
GET /tenants/platform/health	superadmin	tenants + usage_events	Superadmin › Inicio	Tenants activos, consultas hoy, cerca de cuota
GET /tenants/platform/ops	superadmin	conversaciones de cada tenant (paralelo)	Superadmin › Inicio	Colas de espera por tenant, derivaciones de hoy
GET /tenants/platform/system	superadmin	Prometheus (28 queries en paralelo) + /proc + shutil.disk_usage + volumen /backups	Superadmin › Monitoreo › Estado, Métricas, Backups	Servicios up/down, CPU/RAM/disco, PG (conexiones, tamaño, cache hit, deadlocks), Redis (memoria, hit rate, evictions, keys por DB), backend (requests, 5xx, p95 de http_request_duration_highr_seconds), LLM por modelo/estado, contadores ia_*, backups diario/semanal con historial
GET /tenants/platform/alerts	superadmin	Alertmanager API v2	Monitoreo › Estado › "Alertas activas"	Las mismas alertas del email
GET /tenants/platform/errors	superadmin	Redis error buffer	Monitoreo › Errores	Últimos 300 warning/error agrupados
GET /tenants/{id}/health y /{id}/metrics, /{id}/usage	superadmin	usage_events + consultas_log + errores filtrados por tenant + MinIO + tamaño del schema	Superadmin › Tenant › Uso / Salud técnica	Última consulta/ingesta, consultas 7 d por día, latencia p50/p95, cache, documentos por estado, bytes en PG y MinIO

GET /health, /health/ready	público	—	Docker healthcheck, blackbox	Liveness; readiness chequea PG/Redis/Qdrant con timeout 2 s
----------------------------	---------	---	------------------------------	---

7. Dónde se ve cada cosa — mapa de acceso

Superficie	Quién	Acceso	Contenido	Uso real
Email de alertas	2 personas	Resend → Gmail	22 reglas; asunto "🔴 [Intellix] Alerta (n)" / "✅ RESUELTA"	10 emails en la vida del stack
Panel superadmin › Monitoreo	super_admin	intellix.com.ar/superadmin/monitoring (4 tabs: Estado, Métricas, Backups, Errores)	Resumen de Prometheus + host + backups + buffer de errores + alertas activas	Es la vía de uso diario
Panel admin › Métricas	admin del tenant	/admin/metrics/asistente, /atencion	Análítica de negocio del tenant	Visible al cliente (MutuaLyF)
Grafana	equipo Pixs	http://200.58.109.110/grafana/ (HTTP, sin TLS, por IP) o túnel :3001	4 dashboards: IA Platform — Overview (métricas ia_*: 3 de 9 paneles siempre en 0 por Celery), Estado del VPS , Histórico comparado , Logs por contenedor	Usuario equipo nunca inició sesión; admin activo
Prometheus / Alertmanager / Jaeger UI	operador	Túnel SSH :9090 / (9093 no está mapeado en compose, el runbook lo asume) / :16686	Crudo	Diagnóstico
Superadmin › Auditoría	super_admin	/superadmin/audit	audit_log global con filtros por org/acción/fecha	Trazabilidad de acciones humanas

8. Duplicaciones y solapamientos

Dato	Se mide en...	Problema	Recomendación
CPU/RAM por contenedor	cAdvisor y Telegraf	cAdvisor no funciona en este host; aún así corre y se scrapea	Borrar cAdvisor del compose y de prometheus.yml
CPU/RAM/disco del host	node-exporter (Prometheus) y _server_status() / _disk_status() leyendo /proc y shutil desde el backend	Dos fuentes; la del backend existe para que funcione en dev sin Prometheus	Aceptable. Dejar documentado que el panel no depende de node-exporter
Estado de backups	Textfile mutualyf_backup_* (alertas) y _backups_status() leyendo el volumen /backups (panel)	Dos criterios de "vencido": 36 h en la alerta vs 26 h en el panel	Aceptable; unificar el umbral
Latencia de consulta	ia_query_duration_ms (Prometheus, por-worker) · http_request_duration_* (instrumentator) · consultas_log.latency_ms (PG) · spans de Jaeger	Cuatro fuentes; solo la de PG es exacta y persistente. El superadmin muestra el p95 del instrumentator (mezcla todos los endpoints), el admin el p95 de PG (solo consultas)	Fuente de verdad = consultas_log. Prometheus para alertar, no para reportar
Errores del backend	Redis error buffer · Loki · docker logs · http_requests_total{status="5xx"}	El buffer es el único que se mira; Loki está incompleto	Arreglar Loki (o apagarlo) y limpiar el ruido del buffer
Consultas por tenant	ia_queries_total (Prometheus) · usage_events (PG) · consultas_log (PG)	Prometheus por-worker y se resetea en cada restart; PG es la verdad (4.380 vs 10)	Ídem
UIs de administración	pgAdmin · Portainer · Grafana · panel superadmin	pgAdmin y Portainer no se usan (3 meses up, 0 % CPU) y Portainer tiene el wizard vencido	Parar ambos (docker compose stop pgadmin portainer); levantar bajo demanda
Dashboards de Grafana vs panel superadmin › Métricas	Ambos leen Prometheus	Muestran lo mismo con distinta cara; el panel superadmin además mezcla PG/Redis/backups	Grafana para el equipo técnico (histórico), panel para operación diaria. No es duplicación dañina

9. Cuánto consume del VPS

Host: 16 vCPU, 31 GB RAM, 238 GB NVMe (66 GB usados, 28 %), load 0,14–0,32, 102 días de uptime. Lo que sigue es el promedio de 7 días por contenedor (Telegraf). Los porcentajes de CPU son sobre **un** core.

Bloque	Contenedores	CPU (Σ % de 1 core)	RAM	Disco	Red destacada
Métricas + alertas (núcleo)	prometheus, alertmanager, grafana, node-exporter, telegraf, blackbox, postgres-exporter, redis-exporter	5,6 %	360 MB	1,4 GB (TSDB 90 d) + 1 MB Grafana	Prometheus 21 GB rx acumulado; postgres-exporter 27 GB rx (queries de stats cada 15 s)
Logs	loki, promtail	2,5 %	219 MB	1,2 MB	Loki escribió 2 GB a disco en total por compactación (dato de docker stats BlockIO), pero conserva 1,2 MB
Trazas	jaeger	0,9 %	2.008 MB	0 (memoria)	—
Muerto / sin uso	cadvisor, pgadmin, portainer	0,2 %	176 MB	0,8 MB	—
Total observabilidad + admin	14 contenedores	9,3 %	2.763 MB (8,7 % del host)	1,5 GB (0,6 % del disco)	
Referencia: la plataforma en sí (prod)	backend, celery ×2, postgres, qdrant, redis, minio, nginx, frontend	~2,5 %	~2.640 MB	—	nginx 10,8 GB rx / 11,5 GB tx
Referencia: staging completo	7 contenedores *_staging	~2,5 %	~2.270 MB	—	

Lectura: la observabilidad consume **más RAM que la plataforma de producción entera** (2,76 GB vs ~2,64 GB), y el 73 % de eso es Jaeger guardando trazas de /health. En CPU y disco es irrelevante. Apagar Jaeger, cAdvisor, pgAdmin y Portainer baja el bloque a ~580 MB sin perder ninguna capacidad que se esté usando hoy.

Logs Docker (json-file): 95 MB en total; los más grandes son nginx 17 MB, cadvisor 13 MB, celery_worker_staging 12 MB. El backend de prod: 4,4 MB, de los cuales el ~98 % son líneas GET /metrics.

10. Qué está mal — detalle y corrección propuesta

#	Problema	Dónde	Corrección	Esfuerzo
1	/metrics público	nginx/nginx.prod.conf :152 y :274	Eliminar ambos location /metrics (Prometheus llega por backend:8000 en la red interna). Verificar con curl -I https://intellix.com.ar/metrics → 404. Aplicar también en staging por consistencia aunque hoy dé 307	10 min
2	Contadores por-worker	core/metrics.py, entryptoint.sh (--workers 4)	Activar PROMETHEUS_MULTIPROC_DIR=/tmp/prom (tmpfs), registrar MultiProcessCollector en setup_metrics, limpiar el dir en el entryptoint. Los histogramas y gauges necesitan multiprocess_mode. Test: 3 scrapes seguidos deben ser monótonos	2–3 h + deploy
3	Loki descarta lo no-JSON	observability/promtail/promtail.yml	Quitar drop_malformed: true (las líneas no-JSON pasan sin labels extra) y usar match por service para aplicar el stage json solo al backend. Ajuste de 3 líneas; Loki empieza a recibir nginx/celery/postgres al instante	15 min

4	Jaeger: 2 GB de trazas inútiles	.env (OTEL_ENABLED), compose	Corto plazo: OTEL_ENABLED=false en prod y staging + docker compose stop jaeger (force-recreate del backend para releer el .env). Si se quiere conservar: FastAPIInstrumentor.instrument_app(app, excluded_urls="health,metrics"), sampler 100 % solo para el widget, y --memory.max-traces=5000	10 min / 1 h
5	cAdvisor zombi	docker-compose.yml (VPS, sin commitear), prometheus.yml job cadvisor	docker compose rm -sf cadvisor; borrar el job. Ya está borrado en dev-local: hace falta commitear/reconciliar el compose del VPS	10 min
6	Ruido GET /metrics en logs	core/logging_config.py _HealthCheckFilter	Extender el filtro a "GET /metrics". Baja el log del backend un 98 %	5 min
7	ia_active_tenants siempre 0	core/metrics.py :64; panel superadmin "Organizaciones activas"	O setearlo (p. ej. en /platform/health con el count de PG) o eliminar la métrica y que el panel use active_tenants de PG que ya calcula	20 min
8	Métricas de Celery invisibles	workers/ingest_tasks.py	Opción A (simple): dejar de incrementar métricas Prometheus en Celery y derivar el dashboard a usage_events/documentos.status (PG ya tiene todo). Opción B: exponer /metrics en el worker con prometheus_client.start_http_server y agregar el target	1 h
9	Buffer de errores contaminado	core/error_buffer.py, core/security.py	Bajar jwt_decode_failed (token vencido) a INFO; no intentar model_warmup si EMBEDDING_PROVIDER != local; agregar botón "Limpiar" (DEL de la key) al panel	30 min
10	Drift repo ↔ VPS	/opt/mutualyf vs main	Commitear en dev-local ya está (392e57c); falta pasar a dev/main cuando el equipo acuerde la subida, y en el VPS hacer git stash/checkout tras el pull para que el compose vivo sea el del repo. Mientras tanto, cualquier git pull en prod puede fallar por conflicto	Según ventana de deploy
11	Grafana por HTTP y por IP	GF_SERVER_ROOT_URL=http://200.58.109.110/grafana/	La contraseña viaja en claro. Ponerlo detrás de un hostname con TLS (p. ej. grafana.intellix.com.ar) o volver a solo túnel	30 min
12	Alerta 5xx sensible a restarts	rules/alerts.yml HighErrorRate5xx	Añadir mínimo de tráfico y/o excluir la ventana de deploy con un silence automático en deploy.sh	15 min
13	Nomenclatura "Groq" en alertas, runbook y métricas	alerts.yml, RUNBOOK.md, ia_groq_requests_total	Renombrar a ia_llm_requests_total y alertas LlmDownOrNoCredit en un solo cambio coordinado (métrica + regla + dashboard + panel)	1 h

11. Qué falta (no está medido hoy)

- **Trust gate en Prometheus:** no hay contador de answer/refuse ni de uso del juez LLM. En PG sí está desde la 055 (trust_action), pero solo el 24 % de las filas de 30 d lo tiene y ninguna es refuse — vale revisar que el refuse efectivamente se persista (los rechazos honestos existen en la suite).
- **Derivaciones (handoff):** no hay métrica ni alerta "afiliado esperando > N minutos sin operador". El endpoint /platform/ops lo calcula, pero solo cuando alguien abre el Inicio del superadmin. Es el peor escenario del producto y hoy no notifica.
- **Costo del LLM:** usage_events.llm_tokens existe (18,2 M tokens de mutualyf) pero no se convierte a pesos/dólares ni se alerta por consumo diario anómalo. Con gpt-4o-mini es barato, pero un loop o un abuso del widget se vería recién en la factura.
- **WhatsApp:** no hay métrica de mensajes entrantes/salientes ni de errores de la API de Meta (solo un logger.error).
- **Qdrant:** se scrapea pero no hay ni un panel ni alerta (colecciones, puntos, latencia de búsqueda).
- **Celery beat:** si beat muere, las purgas, el chequeo de inactividad de operadores y el escaneo de contradicciones dejan de correr en silencio. No hay heartbeat.
- **Frontend:** cero telemetría (sin Sentry/analytics). Un error de JS en el widget del afiliado es invisible.
- **Retención de usage_events, audit_log, conversaciones/mensajes:** sin purga. Hoy son MB, pero es la tabla más grande del tenant (mensajes 3,3 MB) y crece con cada afiliado.
- **Status page / uptime externo:** blackbox corre dentro del VPS; si el VPS cae, nadie avisa. docs/STATUS_PAGE.md lo propone (UptimeRobot) y no está hecho.

12. Plan sugerido

Prioridad	Acción	Ganancia	Riesgo
Hoy	Quitar location /metrics de nginx prod (y staging), nginx -s reload	Cierra la fuga de datos internos	Ninguno: Prometheus no pasa por nginx
Hoy	OTEL_ENABLED=false + parar Jaeger; parar cAdvisor, pgAdmin, Portainer; quitar job cadvisor	~2,2 GB RAM, ~4 contenedores, menos ruido en Prometheus	Ninguno funcional; Jaeger vuelve con un flag si hace falta
Esta semana	Promtail sin drop_malformed; filtro de /metrics en el access log; bajar jwt_decode_failed a INFO; no hacer warmup con OpenAI	Loki pasa a servir de verdad; el buffer de errores muestra errores	Bajo
Esta semana	Alertas: HandoffSinAtender (nueva, sobre una métrica que el backend exponga o sobre /platform/ops), mínimo de tráfico en 5xx, heartbeat de beat	Cubre el escenario de negocio más grave	Bajo
Próximo sprint	Multiproceso en prometheus_client; Celery → PG en vez de Prometheus (o worker con /metrics); setear/eliminar ia_active_tenants; renombrar Groq → LLM	Métricas correctas y honestas; dashboards sin paneles en 0	Medio (tocar el arranque del backend; probar en staging)
Próximo sprint	Reconciliar compose/observability del VPS con el repo (pasar 392e57c a dev/main); Grafana con TLS; UptimeRobot externo	Deploys sin sorpresas; acceso seguro; aviso si el VPS entero cae	Bajo
Después	Costo LLM en pesos + alerta de consumo; métricas WhatsApp y Qdrant; telemetría del widget; retención de mensajes	Visibilidad de negocio completa	—

Anexo A — Cómo se verificó cada dato

```
# Host y contenedores
ssh -i ~/.ssh/mutualyf_vps -p 2251 root@200.58.109.110
docker ps; docker stats --no-stream; free -h; df -h /
# Prometheus
curl -s localhost:9090/api/v1/targets | jq '.data.activeTargets[] | {job:.labels.job, health}'
curl -s localhost:9090/api/v1/rules | jq ' [.data.groups[].rules[].name] | length' # 22
curl -s localhost:9090/api/v1/status/tsdb | jq .data.headStats # 6887 series
curl -s 'localhost:9090/api/v1/query?query=count_over_time(ALERTS{alertstate="firing"}[30d])'
# Bug multiproceso (repetir 3 veces)
docker exec ia_backend curl -s localhost:8080/metrics | grep '^ia_queries_total'
# Consumo 7 días por contenedor (Telegraf)
avg_over_time(docker_container_mem_usage{container_name="ia_jaeger"}[7d])
# Loki
curl -s localhost:3100/loki/api/v1/label/container/values # ["ia_backend","ia_jaeger"]
curl -s -G localhost:3100/loki/api/v1/query --data-urlencode 'query=sum by (container) (count_over_time({container=~".+"}[7d]))'
docker exec ia_backend curl -s http://promtail:9080/metrics | grep promtail_dropped_entries_total
# Jaeger
curl -s 'localhost:16686/api/traces?service=ia-platform-backend&lookback=24h&limit=200'
# Alertmanager
docker exec ia_backend curl -s http://alertmanager:9093/metrics | grep 'alertmanager_notifications_total{integration="email"}'
# Redis error buffer
docker exec ia_redis redis-cli -n 1 LLEN platform:recent_errors
# PostgreSQL (schema por tenant)
SELECT count(*), percentile_cont(0.5) WITHIN GROUP (ORDER BY latency_ms) FROM tenant_mutualyf.consultas_log WHERE created_at > now()-interval '30 days';
SELECT tenant_id, event_type, count(*), sum(value) FROM usage_events GROUP BY 1,2;
# Exposición pública
curl -s -o /dev/null -w '%{http_code}' https://intellix.com.ar/metrics # 200
# Drift
cd /opt/mutualyf && git status --short && git diff --stat
```

Anexo B — Archivos del repo involucrados

Archivo	Rol
backend/core/metrics.py	Definición de las 8 métricas ia_* + instrumentator HTTP; expone /metrics
backend/core/tracing.py	OpenTelemetry → Jaeger (OTLP gRPC 4317), sampler 10 %
backend/core/logging_config.py	structlog JSON en prod, silencio de librerías, filtro de /health
backend/core/error_buffer.py	Handler de logging → Redis (300 entradas, TTL 7 d)
backend/core/prometheus.py	Cliente HTTP de Prometheus para el panel superadmin (28 queries)

backend/api/v1/metrics.py	Dashboard del admin del tenant (PG)
backend/api/v1/tenants.py (1505–2330)	/platform/health system alerts errors ops, /{id}/health metrics usage, helpers de disco/servidor/backups
backend/services/orchestrator.py (955–1030)	Escritura de consultas_log (con trust_*) y usage_events
backend/workers/cleanup_tasks.py	Purga diaria de consultas_log (180 d) y tool_call_audit (90 d)
observability/prometheus/prometheus.yml, rules/alerts.yml, rules/contenedores.yml	Targets y 22 reglas
observability/alertmanager/alertmanager.yml	Email vía Resend, agrupación, inhibición
observability/promtail/promtail.yml, loki/loki.yml	Pipeline de logs (con el drop_malformed) y retención 30 d
observability/telegraf/telegraf.conf	Métricas por contenedor (solo en dev-local y en el VPS sin commitear)
observability/grafana/dashboards/*.json	4 dashboards provisionados
frontend/app/(superadmin)/superadmin/monitoring/**, components/admin/metrics-dashboard.tsx	Pantallas
docker-compose.yml, docker-compose.prod.yml	Servicios de observabilidad; pgAdmin/Portainer solo en prod
scripts/mutualyf-pg-backup.sh, satellites-backup.sh	Crons del host que publican métricas de backup
observability/RUNBOOK.md, docs/OPERACIONES.md §9	Procedimientos (referencian Jaeger y "Groq")